15/6/25

HW-1 Answers

1) a)

Given two Boolean functions

$\Rightarrow f, g : \{-1, 1\}^n \to \{-1, 1\}$

$\Rightarrow$ Distribution D over inputs $x \in \{-1, 1\}^n$

To Prove:

$$\Rightarrow \Pr_{x\sim D}[f(x) \neq g(x)] = \frac{1 - E_{x\sim D}[f(x)g(x)]}{2}$$

Step 1:

Since $f(x), g(x) \in \{-1, 1\}$,

$\Rightarrow f(x) = g(x) \Rightarrow f(x)g(x) = 1$

$\Rightarrow f(x) \neq g(x) \Rightarrow f(x)g(x) = -1$

Step 2:

Indicator function,

$$I_{f(x)\neq g(x)} = \begin{cases} 1 & \text{if } f(x) \neq g(x) \\ 0 & \text{if } f(x) = g(x) \end{cases}$$

because we are interested in the probability of $f(x) \neq g(x)$.

Now, if we need to express indicator function using $f(x)g(x)$, then

$$I_{f(x)\neq g(x)} = \frac{1 - f(x)g(x)}{2}$$

Since,

~~if f(x)g(x)~~

if $f(x) = g(x)$, $f(x)g(x) = 1$

$\Rightarrow I_{f(x)\neq g(x)} = \frac{1-1}{2} = 0$

if $f(x) \neq g(x)$, $f(x)g(x) = -1$

$\Rightarrow I_{f(x)\neq g(x)} = \frac{1-(-1)}{2} = 1$

Step 3:

So probability that $f(x) \neq g(x)$ under D is,

$$\Pr_{x \sim D}[f(x) \neq g(x)] = E_{x \sim D}[I_{f(x) \neq g(x)}]$$

$$= E_{x \sim D}\left[\frac{1 - f(x)g(x)}{2}\right]$$

Thus

$$\Pr_{x \sim D}[f(x) \neq g(x)] = \frac{1 - E_{x \sim D}[f(x)g(x)]}{2}$$

Hence proved.

— X —

1)b) Yes, the identity still holds true even if the domain changes from $\{-1, 1\}^n$ to R^n, as long as,

$$f, g : R^n \to \{-1, 1\}$$

i.e., the output remains boolean, the same.

Since,

$$\Pr_{x \sim D}[f(x) \neq g(x)] = \frac{1 - E_{x \sim D}[f(x)g(x)]}{2}$$

depends only on the outputs, not on the domain.

— X —

2)

Given,

→ Decision tree function $\left.\right\}$ $f : \{-1, 1\}^n \to \{-1, 1\}$ with t leaves

→ Each leaf corresponds to a path of conditions on input variables, $x_1, \dots x_n$.

To construct,

$p(x_1, \dots x_n)$ such that, $f(x) = p(x)$ for all $x \in \{-1, 1\}^n$

Step 1:

We build one polynomial term per leaf.

→ If path tests,

$x_i = 1$, we include $\frac{1+x_i}{2} = 1$

→ If path tests,

$x_i = -1$, we include $\frac{1-x_i}{2} = 1$

So, for a leaf 'l', the indicator polynomial is,

$$P_l(x) = \prod_{i \in pos(l)} \frac{1+x_i}{2} \cdot \prod_{j \in neg(l)} \frac{1-x_j}{2}$$

where,

→ 'l' denotes leaf

→ pos(l) indicates the indices of variables that are 1 in path l

→ neg(l) indicates the indices of variables that are -1 in path l.

Now, if

Output of leaf l, $f_l \in \{-1, 1\}$. Then

$$p(x_1, \ldots x_n) = \sum_{l=1}^{t} f_l \cdot P_l(x)$$

$$\Rightarrow p(x_1, \ldots, x_n) = \sum_{l=1}^{t} f_l \cdot \left(\prod_{i \in pos(l)} \frac{1+x_i}{2} \cdot \prod_{j \in neg(l)} \frac{1-x_j}{2} \right)$$

—— X ——

3) To build full binary decision tree of depth 2, using Gini impurity function $C(a) = 2a(1-a)$ from given table data.

Step 1:

We need to find the root node among X, Y, Z, which has the lowest Gini impurity C_a.

Lets calculate for X, Y, Z one by one.

For X:

X=0

Total Pos = 10+25+35+35 = 105

Total Neg = 20+5+15+5 = 45

Total = 105+45 = 150

$$C = 2 \cdot \frac{105}{150} \cdot \frac{45}{150} = 0.42$$

X=1

Total Pos = 5+30+10+15 = 60

Total Neg = 15+10+10+5 = 40

Total = 60+40 = 100

$$C = 2 \cdot \frac{60}{100} \cdot \frac{40}{100} = 0.48$$

Weighted Gini for X:

$$C = \left(\frac{150}{150+100} \cdot 0.42\right) + \left(\frac{100}{150+100} \cdot 0.48\right)$$

$$\Rightarrow C = 0.444$$

For Y:

Y=0

Total Pos = 10+25+5+30 = 70

Total Neg = 20+5+15+10 = 50

Total = 70+50 = 120

$$C = 2 \cdot \frac{70}{120} \cdot \frac{50}{120} = 0.4861$$

Y=1

Total Pos = 35+35+10+15 = 95

Total Neg = 15+5+10+5 = 35

Total = 95+35 = 130

$$C = 2 \cdot \frac{95}{130} \cdot \frac{35}{130} = 0.3935$$

Weighted Gini for Y:

$$C = \left(\frac{120}{120+130} \cdot 0.4861\right) + \left(\frac{130}{120+130} \cdot 0.3935\right)$$

$$\Rightarrow C = 0.438$$

For Z:

Z=0

Total Pos = 10+35+5+10 = 60
Total Neg = 20+15+15+10 = 60
Total = 60+60 = 120

$$C = 2 \cdot \frac{60}{120} \cdot \frac{60}{120} = 0.5$$

Z=1

Total Pos = 25+35+30+15 = 105
Total Neg = 5+5+10+5 = 25
Total = 105+25 = 130

$$C = 2 \cdot \frac{105}{130} \cdot \frac{25}{130} = 0.3107$$

Weighted Gini for Z:

$$C = \left(\frac{120}{120+130} \cdot 0.5\right) + \left(\frac{130}{120+130} \cdot 0.3107\right)$$

$$\Rightarrow C = 0.402$$

Among these three weighted gini impurities, as Z has the lowest, we would split based on Z in the root node.

Step 2: Fix Root as Z, Split children on best remaining feature.

-> Left child (Z=0) -> Split on X or Y
-> Right child (Z=1) -> Split on X or Y.

I) Left child (Z=0)

i) For X

~~Total Pos~~

X	Pos	Neg	Tot
0	10+35=45	20+15=35	80
1	5+10=15	15+10=25	40
Total	60	60	120

Weighted Gini for X:

$$\cancel{C = \left(\frac{80}{120} \cdot 2 \cdot \frac{80}{120} \cdot \frac{40}{120}\right) + \left(\frac{40}{120} \cdot 2 \cdot\right.}$$

Weighted Gini for X:

$$C = \left(\frac{80}{120} \cdot 2 \cdot \frac{45}{80} \cdot \frac{35}{80}\right) + \left(\frac{40}{120} \cdot 2 \cdot \frac{15}{40} \cdot \frac{25}{40}\right)$$

$$\Rightarrow C = 0.4844$$

ii) For Y

Y	Pos	Neg	Tot
0	10+5=15	20+15=35	50
1	35+10=45	15+10=25	70
Total	60	60	120

Weighted Gini for Y:

$$C = \left(\frac{50}{120} \cdot 2 \cdot \frac{15}{50} \cdot \frac{35}{50}\right) + \left(\frac{70}{120} \cdot 2 \cdot \frac{45}{70} \cdot \frac{25}{70}\right)$$

$$C = 0.4429$$

So, split left child (Z=0) based on Y.

II) Right Child (Z=1)

i) For X

X	Pos	Neg	Tot
0	25+35=60	5+5=10	70
1	30+15=45	10+5=15	60
Total	105	25	130

Weighted Gini for X:

$$C = \left(\frac{70}{130} \cdot 2 \cdot \frac{60}{70} \cdot \frac{10}{70}\right) + \left(\frac{60}{130} \cdot 2 \cdot \frac{45}{60} \cdot \frac{15}{60}\right)$$

$$C = 0.3049$$

ii) For Y

Y	Pos	Neg	Tot
0	25+30=55	5+10=15	70
1	35+15=50	5+5=10	60
Total	105	25	130

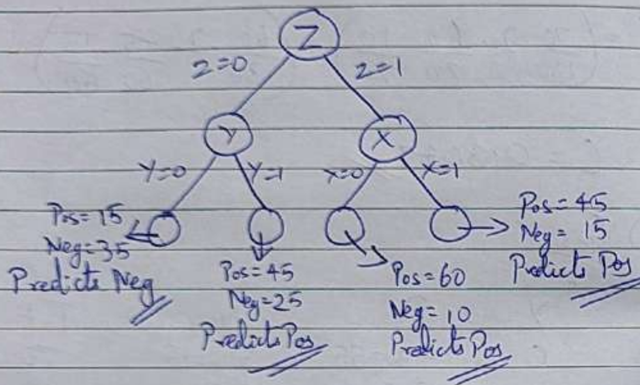
Weighted Gini for Y:

$$C = \left(\frac{70}{130} \cdot 2 \cdot \frac{55}{70} \cdot \frac{15}{70}\right) + \left(\frac{60}{130} \cdot 2 \cdot \frac{50}{60} \cdot \frac{10}{60}\right)$$

$$C = 0.3095$$

So, split right child (Z=1) based on X.

So the final tree is



Step 3: Calculating Accuracy

X	Y	Z	Pos	Neg	Pred	
0	0	0	10	20	Neg	✓
0	0	1	25	5	Pos	✓
0	1	0	35	15	Pos	✓
0	1	1	35	5	Pos	✓
1	0	0	5	15	Neg	✓
1	0	1	30	10	Pos	✓
1	1	0	10	10	Pos	✓
1	1	1	15	5	Pos	✓

So the overall training accuracy $\}$ = 100%

—— X ——

4) Given,

⇒ Domain $X = R$
⇒ Labels $Y = \{-1, 1\}$

⇒ Concept Class $C = h_\theta(x) = \begin{cases} -1 & \text{if } x \le \theta \\ 1 & \text{if } x > \theta \end{cases}$ for $\theta \in R$

To learn a hypothesis, $h_\theta \in C$

with error at most ϵ
with probability at least $1-\delta$
using only $m = O\left(\frac{1}{\epsilon} \log \frac{1}{\delta}\right)$ training examples

⇒ Here, the threshold functions of the concept class C are on the real line.

So VC dimension is 1

PAC Learning Algorithm

1) Input:

Training Set $S = \{(x_1, y_1), \dots, (x_m, y_m)\}$
where $x_i \in R$
$y_i \in \{-1, 1\}$

2) Sort the training examples by x_i.

3) Now we need to find where the labels y_i change from -1 to $+1$, i.e., threshold between largest negative example & smallest positive example.

Let largest negative example $= x_{neg}^{max}$

Smallest positive example $= x_{pos}^{min}$

$$\Rightarrow x_{neg}^{max} = \max\{x_i \mid y_i = -1\}$$

$$x_{pos}^{min} = \min\{x_i \mid y_i = 1\}$$

$\Rightarrow$ Threshold θ is choosen such that

$$x_{neg}^{max} \leq \theta < x_{pos}^{min}$$

4) This gives us the hypothesis

$$h_\theta(x) = \begin{cases} -1 & \text{if } x \leq \theta \\ 1 & \text{otherwise} \end{cases}$$

If there exists a threshold that clearly separates the data, then this algorithm finds a clear & consistent classifier.

Here, as the VC dimension is 1,

from standard PAC bounds,

to get error $\leq \epsilon$ & probability $\geq 1-\delta$,

Training example size $\Big\}$ $m \geq \frac{1}{\epsilon} \log \frac{1}{\delta}$

This is the required PAC learning algorithm

—— X ——

5) The final goal is to show that,

a mistake bounded learner can be Converted into a PAC learner.

Given,

-> Mistake bounded algo A with mistake bound t.

-> Domain $X = \{-1, 1\}^n$,

-> Labels $Y = \{-1, 1\}$

-> To design PAC learning algorithm B, that uses A as a subroutine.

To prove,

$\forall \delta, \epsilon \in [0,1], \exists m \in N$ such that

$$\Pr_{S \sim D^m} [err(B(S)) > \epsilon] \leq \delta$$

a)

a) Fix hypothesis h' & bound k.

-> To find,

how many examples are needed to ensure if h' classifies them all Correctly, then 'true error' is small.

Let $err(h') = P_{x \sim D}[h'(x) \neq c(x)]$

Let us assume $err(h') > \epsilon$, then

probability that h' gets all k examples correct is at most,

$$(1-\varepsilon)^k \leq e^{-\varepsilon k}$$

So if

$$e^{-\epsilon k} \leq \delta' \Rightarrow k \geq \frac{1}{\varepsilon} \log\left(\frac{1}{\delta'}\right)$$

So if h' labels $k \geq \frac{1}{\varepsilon} \log\left(\frac{1}{\delta'}\right)$ examples correctly

$\Rightarrow \Pr[err(h') > \varepsilon] \leq \delta'$

b) Let us say algorithm B runs A & makes at most 't' mistakes.

→ We want to ensure that with high probability all 't' hypothesis have error at most ϵ.

→ Failure probability $\delta' = \frac{\delta}{t}$

Based on (a), $K \geq \frac{1}{\epsilon} \log\left(\frac{t}{\delta}\right)$

→ Since A at most makes 't' mistakes,

we need at most 't' of these K-sized clean blocks, so that our hypothesis holds true.

So, $m = t \cdot K = \frac{t}{\epsilon} \log\left(\frac{t}{\delta}\right)$

c) PAC learner B with A as subroutine.

Pseudocode:

1) Input: ϵ, δ, Distribution D, mistake bound learner A with mistake bound 't'.

2) Initialize A.

3) Draw $m = \frac{t}{\epsilon} \log\left(\frac{t}{\delta}\right)$ labeled examples from D

4) For each (x, y):
   - Predict using A
   - If wrong, update A.

5) Output the final hypothesis h returned by A.

Proof.

→ With probability $\geq 1-\delta$, each hypothesis that made $\leq K$ mistakes will have true error $\leq \epsilon$, as long as it classifies atleast

$K = \frac{1}{\epsilon} \log\left(\frac{t}{\delta}\right)$ clean examples correctly

Since A makes $\leq t$ mistakes, there will be at most t hypotheses.

To be confident, we apply union bound & draw

$$m = t \cdot \frac{1}{\varepsilon} \log\left(\frac{t}{\delta}\right) \text{ examples.}$$

So, for any $\varepsilon, \delta \in (0,1)$, the output hypothesis from B has,

$\Rightarrow$ True error $\leq \varepsilon$

$\Rightarrow$ Probability $\geq 1-\delta$

Hence Algorithm B is a PAC Learner

So Concept Class C that has mistake bound algorithm A with bound t is PAC-learnable with,

$$m = O\left(\frac{t}{\varepsilon} \log\left(\frac{t}{\delta}\right)\right)$$

— X —